that when they are loaded they refer to each other, then all the modules will be loaded at the start time, which will kill the basic aim of a modular designed server. To solve this problem, GlassFish uses a technique by which programmers are prevented from directly programming with modules. GlassFish uses a concept of service. These services are classes that implement an interface. And programmers are encouraged to program on these well-defined services instead of programming with modules.

Persistence

Persistence is one of the most critical service given by an application server to the applications running on it. GlassFish has a lot of design challenges for its persistence service, mainly due to the old issues involved in the persistence mechanism. Java EE programmers have always seen various implementations of ***javax.persistence api***, which have issues with the old JDBC classes and their usage.

According to the GlassFish wiki, the following patterns were identified while designing persistence in GlassFish:

1. Applications can do a ***Class.forName( )*** call to load a JDBC driver and make direct calls through the JDBC APIs to the underlying database. Applications do not need to identify a special interest in such drivers. There is also no indication they will use such low level access APIs.
2. Applications can decide to use the default JPA provider, letting GlassFish choose which one to use.
3. In Java SE mode, applications can use the ***PersistenceProvider.createEntityManagerFactory()*** call to get a specific persistence provider (name extracted from the ***persistence.xml***).
4. In Java EE mode, GlassFish is responsible for loading the ***persistence.xml*** file, and looking up the expected persistence provider according to its settings (potentially defaulting the provider name).
5. In Java EE mode, GlassFish is responsible for identifying the connection pool to be used with the entity manager. This resource adapter is retrieved at deployment time from the JNDI name space and wired up with the persistence provider to create the entity manager.

Container pluggability

Containers are the heart of an application server because they are the entities that run an application. For example, the EJB container in an application server is responsible for running Java EJB applications. This EJB container provides various additional features for Bean developers like security, scalability, client interaction and a messaging service. Thus, an application programmer is only concerned with implementing the business logic; all the other stuff is handled by the container. Containers are always created as pluggable units, such that they can be installed or removed from the application server.

According to the GlassFish wiki: "Containers have the ability to install themselves in an existing GlassFish installation with the following services implementations:

1. ***Sniffer:*** Invoked during a deployment operation (or server restart). Sniffers will have the ability to recognise part (or whole) application bundles. Once a sniffer has positively identified parts of the application, its set-up method will be called to set up the container for use during that server instance lifetime.
2. ***ContainerProvider:*** This is called each time a container is started or stopped. A container is started once the first application that's using it is deployed. It will remain in operation until it is stopped when the last application using it has been stopped.
3. ***Deployer:*** This is called to deploy/undeploy applications to a container.
4. ***AdminCommand:*** Containers can come with a special set of CLI commands that should only be available once the container has been successfully installed in a GlassFish v3 installation. These commands should be used to configure the container's features."

Winding up

The promise of the GlassFish community is to make a production-quality application server and add new features to it faster than even before. The following are the key features of the GlassFish Application server:

1. Production quality
2. Robust
3. Scalable
4. Secure
5. Has load balancing
6. Delivers more work with less code
7. Issue tracking
8. Service-oriented architecture.
9. Tools integration
10. Ease of container pluggability

And one of the most important features is that, being an open source project, we have access to the source code and can understand the bits and bytes of an enterprise-class application server.

In my next article I will explain how to get started with GlassFish. Additionally, I will also develop a simple Java enterprise application using GlassFish. END

References

- Wikipedia: *en.wikipedia.org/wiki/Application_server*
- GlassFish home page: *glassfish.dev.java.net*
- GlassFish wiki: *wiki.glassfish.java.net*
- The v3 Engineers' Guide: *wiki.glassfish.java.net/Wiki.jsp?page=V3EngineersGuide*

By: Rajeev Kumar

The author is a Software Engineer working at Aricent Technologies. He loves working with GNU/LINUX and FOSS in general, as well as Java Enterprise Edition. You can send your comments or questions to rajeevcoder@yahoo.com